The Infinite Context Protocol: Implementing MIT's Recursive Logic for Business

RECURSIVE LANGUAGE MODELS

Alex L. Zhang
MIT CSAIL
altzhang@mit.edu

Tim Kraska
MIT CSAIL
kraska@mit.edu

Omar Khattab
MIT CSAIL
okhattab@mit.edu

ABSTRACT

We study allowing large language models (LLMs) to process arbitrarily long prompts through the lens of inference-time scaling. We propose **Recursive Language Models** (**RLM**s), a general inference strategy that treats long prompts as part of an external *environment* and allows the LLM to *programmatically* examine, decompose, and recursively call itself over snippets of the prompt. We find that RLMs successfully handle inputs up to two orders of magnitude beyond model context windows and, even for shorter prompts, dramatically outperform the quality of base LLMs and common long-context scaffolds across four diverse long-context tasks, while having comparable (or cheaper) cost per query.

1 INTRODUCTION

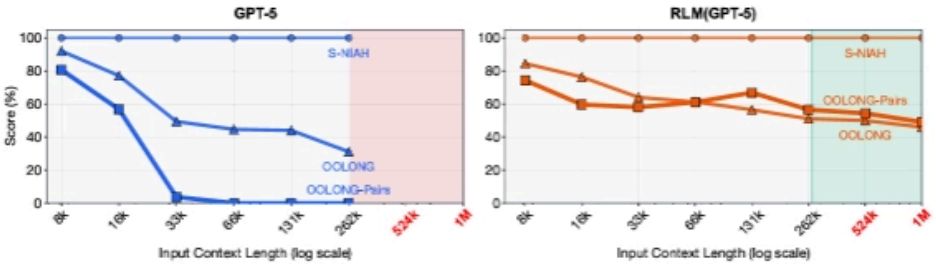

Figure 1: A comparison of GPT-5 and a corresponding RLM on three long-context tasks of increasing complexity: **S-NIAH**, **OOLONG**, and **OOLONG-Pairs**. For each task, we scale the input length from 2^{13} to 2^{18}. GPT-5 performance degrades significantly as a function of both input length and task complexity, while the RLM maintains strong performance. Inputs beyond the red region do not fit in GPT-5's context window of 272K tokens, but the RLM handles them effectively. Additional experiments across other models, methods, and benchmarks are in §2.

Despite rapid progress in reasoning and tool use, modern language models still have limited context lengths and, even within these limits, appear to inevitably exhibit *context rot* (Hong et al., 2025), the phenomenon illustrated in the left-hand side of Figure 1 where the quality of even frontier models like GPT-5 degrades quickly as context gets longer. Though we expect context lengths to steadily rise through improvements to training, architecture, and infrastructure, we are interested in *whether it possible to dramatically scale the context size of general-purpose LLMs by orders of magnitude*. This is increasingly urgent as LLMs begin to be widely adopted for long-horizon tasks, in which they must routinely process tens if not hundreds of millions of tokens.

The Infinite Context Protocol

Implementing MIT's Recursive Logic for Business

How to make AI read, analyze, and recall massive datasets without "Context Rot."

Part 1: The "Context Rot" Crisis

Why Your "Chat with PDF" Workflows Are Failing

You've seen the marketing. Gemini claims 2 million tokens. Claude offers 200k. OpenAI sits at 128k. The promise? "Upload your entire codebase. Chat with your 10-K filings. Analyze your full customer database."

The reality? It's a mirage.

I call this phenomenon **Context Rot** — the silent degradation of AI reasoning as file size grows. Here's what's actually happening inside these "massive context window" models:

The "Goldfish" Effect

AI reasoning quality degrades as the amount of context increases:

- **Small Context (10 pages):** Peak intelligence. The model can recall specific details, cross-reference sections, and provide nuanced analysis.
- **Medium Context (50 pages):** Hallucinations begin. The model starts "filling in gaps" with plausible-sounding but incorrect information.
- **Large Context (100+ pages):** Reasoning collapse. The model can only provide surface-level summaries and misses critical details buried in the middle.

This isn't a bug — it's a fundamental limitation of transformer architecture. The model's "attention" is spread thin, like a flashlight beam trying to illuminate a stadium.

The "One-Shot" Trap

Most users approach AI with a "one-shot" mentality: upload everything at once, ask one question, expect a perfect answer.

But here's the problem: **Complexity doesn't scale linearly. It scales quadratically.**

A 10-page document has ~45 potential cross-references between sections. A 100-page document? Over 4,950. The AI isn't just reading 10x more — it's trying to track 100x more relationships.

The Business Cost of Context Rot

This isn't an academic problem. Real businesses are making real mistakes:

- **Finance:** A hedge fund analyst uploads a company's 10-K filing. The AI summarizes revenue growth but misses the crucial footnote about pending litigation that would materially affect the stock.
- **Tech:** A developer asks the AI to analyze a legacy codebase. The AI confidently explains a function — but hallucinates its parameters because it lost track of the actual definition 500 lines earlier.
- **Legal:** A lawyer uses AI to review a contract. The AI conflates terms from different sections, creating a summary that misrepresents the actual agreement.

In each case, the AI appeared confident. The output looked polished. But the reasoning was rotten.

Part 2: The MIT Solution

The "Recursive Executive" Model

Researchers at MIT's Computer Science and Artificial Intelligence Laboratory (CSAIL) asked a simple question: What if, instead of forcing an AI to be a superhuman reader, we trained it to be a superhuman delegator?

The result is a paradigm shift in how we should interact with AI for complex tasks.

The Old Model: The "Overwhelmed Intern"

Imagine you hand a new intern a 500-page report and say, "Read this and tell me the three most important things."

What happens?

- They skim frantically
- They focus on the beginning and end (recency bias)
- They miss details buried in the middle
- They confidently summarize... but get key facts wrong

This is how most people use AI today. We dump everything into the context window and expect the model to "just figure it out."

The New Model: The "Recursive Executive"

Now imagine a different approach. You hand that same 500-page report to a **Project Manager with a laptop**.

Their process:

1. **Inspect the structure:** They don't read the report. They skim the table of contents, note the sections, and understand the shape of the information.
2. **Write a chunking plan:** They break the report into logical segments — by chapter, by topic, by date range.
3. **Assign sub-agents:** They delegate each chunk to a focused analyst (or, in our case, a focused AI call) with a specific question.
4. **Compile the final report:** They aggregate the findings, resolve conflicts, and synthesize the answer.

This is the MIT breakthrough. Instead of asking AI to be an omniscient reader, we ask it to be an intelligent orchestrator.

The Technical Secret: The REPL Environment

Here's where it gets interesting.

Most AI interfaces (like ChatGPT's default mode) operate in a "conversation" paradigm. You type, the AI responds, the text accumulates in the context window.

But advanced AI systems now offer something more powerful: a **REPL Environment** (Read-Eval-Print Loop).

In a REPL, the AI can:

- **Write code** (Python, SQL, etc.)
- **Run that code** against your uploaded files
- **See the results** of that code
- **Decide what to do next** based on those results

This is the key insight: **The AI doesn't need to "remember" your entire document. It needs to write code that extracts the relevant pieces on demand.**

Think of it like this: A librarian doesn't memorize every book. They know how to use the catalog system to find what you need. The REPL gives AI a catalog system.

The MIT Results

When researchers applied this recursive, code-driven approach:

- **Infinite Context:** There's no practical limit on how much data the AI can process, because it never loads everything at once.
- **Higher Accuracy:** By forcing the AI to extract specific data (rather than "recall" it), accuracy improved by double-digit percentages.
- **Lower Cost:** Using cheap code execution for heavy lifting, rather than expensive LLM inference, dramatically reduced API costs.

Part 3: The "Ghostwriter" Prompt

Your Copy-Paste Implementation

Below is the **exact system prompt** that transforms your AI from an "Overwhelmed Intern" into a "Recursive Executive."

Compatibility: This prompt works with any AI that has code execution capabilities:

- ChatGPT with "Code Interpreter" / "Advanced Data Analysis" enabled
- Claude with "Analysis" mode
- Google Gemini with "Code Execution"

The "Infinite Context" System Prompt

```
> Loading JavaScript code...
```

How to Use This Prompt

1. **Copy the entire prompt** above.
2. **Paste it** at the start of a new chat with your AI (ChatGPT, Claude, Gemini).
3. **Upload your file** (PDF, CSV, codebase, etc.).
4. **Replace the [USER REQUEST]** section with your specific question.
5. **Let the AI work.** It will write code, process your file in chunks, and return a structured answer.

Part 4: Industry Playbooks

7 Workflows to Deploy This Week

Below are specific prompts tailored to common business use cases. Each one follows the Recursive Protocol structure but is optimized for a particular industry pain point.

Playbook 1: Sales — The "RFP Assassin"

The Problem: You receive a 120-page RFP (Request for Proposal) with hundreds of requirements buried in legalese. Missing even one "shall" or "must" could disqualify your bid.

The Prompt:

```
> Loading JavaScript code...
```

Why It Works: Instead of asking the AI to "summarize the RFP" (which triggers Context Rot), you're asking it to run a targeted search for specific language patterns. The AI writes code to scan every page, extract matches, and compile a structured table.

Playbook 2: Product Management — The "Voice of Customer" Engine

The Problem: You have 5,000 customer support tickets, NPS responses, or feedback forms. You need to identify the top 10 recurring themes — but manual analysis would take weeks.

The Prompt:

```
> Loading JavaScript code...
```

Why It Works: The AI doesn't try to "read" 5,000 comments. It writes code to perform statistical text analysis inside the Python environment, then surfaces only the aggregated insights into the chat.

Playbook 3: Finance — The "10-K Analyst"

The Problem: You need to analyze a company's 10-K annual report to assess financial health, but the critical details (debt covenants, off-balance-sheet liabilities, risk factors) are buried in footnotes across 200+ pages.

The Prompt:

```
> Loading JavaScript code...
```

Why It Works: Financial documents are designed to be dense and cross-referenced. By forcing the AI to extract specific terms and flag discrepancies, you catch the details that a surface-level summary would miss.

Playbook 4: Legal — The "Discovery Drone"

The Problem: You're reviewing 50 contracts for a due diligence project. You need to find every clause related to "Change of Control," "Assignment," or "Termination for Convenience" — and flag any non-standard language.

The Prompt:

```
> Loading JavaScript code...
```

Why It Works: Contract review is a pattern-matching problem. The AI iterates through each contract, extracts matching clauses, and applies a simple heuristic (is this language common across the set?) to prioritize human review time.

Playbook 5: Marketing — The "Voice DNA Extraction"

The Problem: You want to train AI to write in your brand's unique voice — but "just read our blog and figure it out" produces generic results.

The Prompt:

```
> Loading JavaScript code...
```

Why It Works: Instead of asking the AI to "learn" your voice (vague), you're asking it to mathematically define your voice (specific). The output becomes a reusable system prompt for all future content.

Playbook 6: Engineering — The "Legacy Code Mapper"

The Problem: You've inherited a codebase with 100,000+ lines of code, minimal documentation, and no one who understands how it all connects. Before you refactor, you need a map.

The Prompt:

```
> Loading JavaScript code...
```

Why It Works: Code is structured data. The AI can parse it programmatically, trace relationships, and surface the 20% of functions that drive 80% of the logic.

Playbook 7: HR & Recruiting — The "Resume Radar"

The Problem: You have 200 resumes for an open role. You need to shortlist the top 10 candidates based on specific criteria — but keyword filtering misses great candidates with non-standard backgrounds.

The Prompt:

```
> Loading JavaScript code...
```

Why It Works: You're not asking the AI to "find the best candidates" (subjective). You're giving it explicit, weighted criteria and asking it to apply those criteria systematically across all files.

Part 5: Troubleshooting & Best Practices

What to Do When Things Go Wrong

Even with the Recursive Protocol, you'll encounter edge cases. Here's how to handle them:

Problem 1: The "Lazy Executive" Error

Symptom: The AI ignores your protocol and immediately tries to summarize the file from memory.

Cause: The AI is defaulting to its "helpful assistant" behavior — trying to give you a fast answer instead of following the process.

Fix: Add this line to the start of your prompt:

```
> Loading JavaScript code...
```

Problem 2: The "Time-Out" Loop

Symptom: The AI's code execution times out or crashes when processing a large file.

Cause: The chunk size is too large, or the file format is causing parsing errors.

Fix: In Step 1, instruct the AI to use smaller chunks:

```
> Loading JavaScript code...
```

Problem 3: The "Cost Spiral"

Symptom: Your API bill skyrockets because the AI is running too many code iterations.

Cause: The Recursive Protocol is powerful — but it uses compute. Processing a 500-page document might require 50 code executions.

Fix: Use the protocol only for high-value tasks (contracts, financials, code) where accuracy justifies the cost. For low-stakes tasks (brainstorming, casual summarization), the standard conversational approach is fine.

Problem 4: The "Human Garnish" Illusion

Symptom: The AI's output is accurate but reads like a data dump. It lacks narrative flow, strategic insight, or creative framing.

Cause: The Recursive Protocol optimizes for extraction and accuracy — not for storytelling or persuasion.

Fix: Separate your workflow into two phases:

1. **Phase 1 (Researcher):** Use the Recursive Protocol to extract facts, data, and quotes.
2. **Phase 2 (Writer):** Paste the extracted data into a new chat and ask the AI to craft it into a narrative, presentation, or report.

The researcher doesn't need to be creative. The writer doesn't need to be accurate. Keep them separate.

Final Thought

The future of AI isn't about who has the biggest memory. It's about who has the best process.

Context windows will keep growing. But human attention will not. The professionals who learn to orchestrate AI — to break problems into chunks, to verify outputs systematically, to use code as a reasoning tool — will outperform those who simply "ask and hope."

Stop accepting hallucinations. Start demanding recursive accuracy.

Your documents deserve better. Now you have the protocol to deliver it.

The Infinite Context Protocol — a LeadPanther framework for recursive AI reasoning.